Bandwidth Usage Monitor and Analyzer: Design and Evaluation of an Integrated Network Telemetry Platform

Project Research Paper | 2026

Abstract

Network usage is often discussed as a single number, even though practical network problems involve several connected questions: which applications consume capacity, when demand rises, whether delay and packet loss change with utilization, and how operators can plan for future demand. This paper presents a locally deployable bandwidth monitoring and analysis platform built with Python, SQLite, and Streamlit. The system collects interface traffic counters, maps some active connections to processes, records latency and packet loss probes, and organizes observations for application, device, and time-based analysis. It also includes anomaly detection, short-term demand forecasting, congestion analysis, and an interactive capacity planner. The repository contains a simulator for producing seasonal traffic and labelled anomalies, alongside a research notebook intended to compare analytical methods. This makes repeatable evaluation possible, but simulated findings should not be treated as measurements of a production network. The contribution is therefore an integrated engineering prototype and an evaluation workflow, rather than a claim that one model or network policy is universally optimal.

Keywords: network monitoring; bandwidth usage; telemetry; anomaly detection; forecasting; congestion

1. Introduction

Reliable network service depends on more than knowing how many bytes moved. A high traffic total may be expected during a scheduled backup, while a smaller but unusual increase may point to a misconfigured application. Latency and packet loss can also affect users even when average throughput appears acceptable. In many small or medium deployments, these measurements are split across operating-system tools, dashboards, and spreadsheets, making it difficult to connect a symptom with its likely source.

The Bandwidth Usage Monitor and Analyzer brings these tasks into one application. Its purpose is to collect or simulate network observations, retain them in a relational store, and make them useful through dashboards and analytical tools. The system is designed primarily for local monitoring and engineering exploration. It is not a packet-content inspection system: encrypted traffic is attributed using connection metadata and process information, not by decrypting payloads.

This paper describes the platform architecture, the data and analytical methods represented in the code, and the limits of the available evaluation. It emphasizes the distinction between implemented capabilities and evidence that would require a real operational deployment.

2. System Design

2.1 Data collection and storage

The live collector samples per-interface byte counters through psutil. A process mapper attempts to associate local network connections with process identifiers and application names. A separate ping probe records round-trip latency, jitter, and packet loss. Optional packet probing is available through Scapy, but it can require elevated operating-system privileges; the application is designed to fall back to socket-level information when raw capture is unavailable.

Collected records are stored in SQLite. The schema covers users, devices, applications, network sessions, upload and download volumes, network quality metrics, alerts, forecasts, settings, anomaly labels, and hourly aggregates. Named parameterized queries and a repository layer separate database access from analytics. Foreign-key checks and write-ahead logging are enabled, while a retention workflow rolls older raw samples into hourly summaries.

2.2 Analytics and user interface

The Streamlit interface presents live status and separate views for applications, devices, time patterns, upload-download balance, rankings, anomalies, forecasting, congestion, efficiency, reports, and settings. Application and device views summarize volume and contribution. Time analysis groups observations by hour, day, and weekday or weekend. The reports page supports exports and replay of historical periods. Budgets and alert rules allow a user to set operational

thresholds.

3. Analytical Methods

3.1 Anomaly detection

Three detection approaches are included: a seasonal rolling Z-score conditioned on hour and day of week; interquartile-range fences, which do not assume normally distributed data; and an Isolation Forest applied to multiple telemetry features. Alerts are assigned information, warning, or critical severity. The project also provides an evaluation path that compares detections with labelled anomalies generated by the simulator. Precision, recall, and F1 score are useful summaries, but their values depend on the simulated labels and the chosen alert threshold.

3.2 Demand forecasting

The forecasting module compares a seasonal-naive baseline with Holt-Winters smoothing and gradient boosting using lagged traffic values. Evaluation is designed as a time-ordered walk-forward split so future observations do not leak into model training. Error is summarized using mean absolute error, root mean squared error, and mean absolute percentage error. Forecasts include a 24-hour horizon and confidence bounds for capacity planning. These methods are decision aids; forecast quality must be checked against a representative local traffic history before operational reliance.

3.3 Congestion and efficiency

The congestion analysis flags periods above an 80 percent utilization threshold, examines Pearson correlations among utilization, latency, jitter, and packet loss, and identifies prominent traffic consumers during congested periods. Its what-if planner lets an operator vary the number of active users and link capacity to estimate changes in saturation. The efficiency score combines useful-throughput share, peak-to-average smoothness, off-peak shifting potential, upload-download balance, and network-quality measures. Because these are composite indicators, the displayed score is best read as a structured summary rather than an objective measure of network quality.

4. Evaluation Approach and Findings

The repository includes a simulator capable of generating up to 90 days of seasonal telemetry and labelled anomalies. A research notebook uses this data to produce figures for application consumption, congestion correlations, anomaly-method comparisons, forecasting, and a traffic-shifting scenario. The forecasting notebook compares predictions over a held-out 72-hour period, while the optimization illustration reduces modeled traffic above the threshold by applying a 22 percent reduction to those hours.

These materials establish a reproducible evaluation workflow, but the figures are not sufficient evidence of performance on a real network. The traffic-shifting result is a modeled scenario, not the outcome of an executed intervention. Likewise, anomaly scores against injected labels measure agreement with the simulator’s rules and do not guarantee detection of naturally occurring incidents. No numeric benchmark values are reported here because they should be regenerated from the exact dataset and environment used for a study.

A suitable field evaluation would run the collector on representative endpoints for several weeks, record known incidents and scheduled workloads, and report data completeness alongside model metrics. Forecasts should be compared against simple baselines on the same time periods. Congestion recommendations should be assessed by measuring utilization and user-facing quality before and after changes, while accounting for changes in demand.

5. Discussion

A key strength of the project is its end-to-end view. Interface counters, process attribution, network-quality measurements, storage, analysis, and reporting are connected in a single local application. The combination of several anomaly methods and multiple forecast baselines also makes the system more inspectable than a dashboard that exposes only one opaque score. The user can explore traffic patterns, investigate periods of high utilization, and test capacity assumptions without changing the network itself.

There are practical limits. Process attribution may be incomplete when operating-system permissions restrict socket inspection. HTTPS encryption prevents payload-level identification, and process names are not always enough to

distinguish specific services. SQLite is appropriate for a local prototype and moderate workloads, but sustained, large-scale multi-site telemetry would benefit from a distributed time-series storage design. Correlation also does not establish that one network measure causes another. Finally, synthetic datasets simplify real traffic and should be treated as test fixtures, not a substitute for operational validation.

6. Conclusion

The Bandwidth Usage Monitor and Analyzer is an integrated prototype for observing and interpreting network usage. It combines collection, local persistence, descriptive analytics, anomaly detection, forecasting, congestion exploration, and reporting in one workflow. Its research notebook provides a starting point for repeatable experiments, while the simulator enables controlled tests. The next meaningful step is validation on real, consented network telemetry: measure collection coverage, establish baseline error rates, and compare recommendations with observed outcomes. With that evidence, the platform can move from a technically broad prototype toward a dependable operational tool.

References

[1] psutil contributors. “psutil documentation.” https://psutil.readthedocs.io/

[2] Scikit-learn developers. “IsolationForest.” Scikit-learn User Guide. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[3] Statsmodels developers. “Exponential Smoothing.” Statsmodels documentation. https://www.statsmodels.org/

[4] Streamlit, Inc. “Streamlit documentation.” https://docs.streamlit.io/

[5] Bandwidth Usage Monitor & Analyzer project repository. README, research notebook, and implementation modules, 2026.